

Personal Finance Manager

Project Documentation

Month 1: Python Programming Mastery — Basic + Standard + Advanced Version

A command-line expense tracking application built in Python

Table of Contents

Table of Contents	2
1. Project Overview and Objectives	4
1.1 Overview	4
1.2 Objectives.....	4
1.3 Project Scope	4
2. Setup and Installation Instructions	5
2.1 Prerequisites	5
2.2 Step-by-Step Installation.....	5
2.3 requirements.txt Contents.....	5
2.4 First Run	5
3. Code Structure Explanation	6
3.1 Folder Layout	6
3.2 Why This Structure?.....	6
3.3 Module-by-Module Explanation	6
expense.py — The Expense Class	6
file_manager.py — Persistence Layer.....	7
utils.py — Input Validation	7
menu.py — Command-Line Interface.....	7
reports.py — Reporting and Analysis (Standard)	7
budget.py — Budget Planning (Advanced)	7
exporter.py — Data Export (Advanced).....	7
charts.py — Chart Generation (Advanced)	7
4. Screenshots of Working Application.....	8
4.1 Main Menu.....	8
4.2 Adding an Expense (Basic)	8
4.3 Viewing All Expenses (Basic)	9
4.4 Category-Wise Summary (Standard)	10
4.5 Monthly Report (Standard).....	11
4.6 Setting a Budget (Advanced).....	13
4.7 Budget Status (Advanced).....	13
4.8 Exporting Data (Advanced)	14
4.9 Generated Spending Chart (Advanced)	15
5. How Technical Requirements Were Met	17

6. User Manual.....	18
6.1 Starting the App	18
6.2 Menu Options	18
6.3 Tips	18
7. Troubleshooting Guide	20
7.1 Testing Evidence	20

1. Project Overview and Objectives

1.1 Overview

The Personal Finance Manager is a command-line Python application that lets a user record, review, and analyze everyday expenses. It was built as the Month 1 capstone project for practicing core Python skills: object-oriented programming, file handling and data persistence, error handling, and modular code organization.

The application stores expense data permanently in a CSV file, so nothing is lost between sessions, and presents an interactive text menu for adding, viewing, searching, and deleting expenses. Beyond the basic tracking features, the project also implements the Standard tier (reporting and analysis) and the Advanced tier (budget planning, data export, and charts), so a single build demonstrates all three project options described in the assignment brief.

1.2 Objectives

- Apply object-oriented design by modeling an expense as a class with clear attributes and behavior.
- Implement reliable file handling and data persistence using Python's built-in csv module.
- Build comprehensive error handling so invalid user input never crashes the program.
- Organize code into logical, reusable modules rather than one large script.
- Extend the basic tracker with reporting/analysis, budget planning, data export, and chart generation.
- Produce complete project documentation, including setup instructions, a user manual, and a code walkthrough.

1.3 Project Scope

Tier	What It Adds
Basic	Add, view, search, and delete expenses; CSV persistence; manual backups.
Standard	Category-wise summary and monthly reports (adds reporting and analysis on top of Basic).
Advanced	Budget planning with over-budget warnings, data export to JSON/Excel, and spending charts (adds charts, budget planning, and export on top of Standard).

2. Setup and Installation Instructions

2.1 Prerequisites

- Python 3.8 or later installed on your computer.
- pip (Python's package installer), which comes bundled with Python.
- A terminal / command prompt.

2.2 Step-by-Step Installation

1. Download or clone the project folder to your computer.
2. Open a terminal and navigate into the project folder:

```
cd finance_manager
```

3. Verify your Python version:

```
python3 --version
```

4. Install dependencies. Basic and Standard features need no installs at all — they use only Python's standard library. The Advanced-tier features (Excel export and charts) need two small packages:

```
pip install -r requirements.txt
```

5. Run the application from the project's root folder:

```
python3 main.py
```

2.3 requirements.txt Contents

```
# Basic & Standard versions use only Python's built-in standard library
# (csv, os, shutil, datetime, json, sys, collections) – nothing to install.
#
# Advanced version features need these two packages:
matplotlib>=3.7      # for generating spending charts
openpyxl>=3.1         # for exporting to Excel (.xlsx) files
```

2.4 First Run

The project ships with five sample expenses already in `data/expenses.csv`, so you can explore every menu option (View, Category Summary, Monthly Report, etc.) immediately without entering data manually first.

3. Code Structure Explanation

3.1 Folder Layout

```
finance_manager/
├── main.py           # Entry point – run this file to start the app
├── requirements.txt  # matplotlib + openpyxl (Advanced tier only)
├── src/
│   ├── expense.py   # Expense class (data model)
│   ├── file_manager.py # CSV read/write/backup functions [Basic]
│   ├── menu.py      # Command-line interface & menu logic
│   ├── utils.py     # Input validation & formatting helpers
│   ├── reports.py   # Category summary & monthly reports [Standard]
│   ├── budget.py    # Budget setting & over-budget checks [Advanced]
│   ├── exporter.py  # JSON / Excel export [Advanced]
│   └── charts.py    # Matplotlib bar chart generation [Advanced]
├── data/
│   ├── expenses.csv # Expense data (sample data included)
│   ├── budgets.csv  # Saved category budgets
│   ├── backups/     # Timestamped CSV backups
│   └── exports/      # JSON / Excel exports
├── reports/         # Generated chart images (.png)
├── docs/
│   └── user_guide.md # End-user menu-by-menu guide
└── screenshots/     # Screenshots of the running app
```

3.2 Why This Structure?

Each responsibility lives in its own module instead of one large script. This mirrors how real-world Python projects are organized and makes the codebase easier to read, test, and extend one piece at a time:

- `expense.py` defines what an expense is (the data model).
- `file_manager.py` handles how data is saved and loaded (persistence).
- `utils.py` holds small, reusable helper functions (validation/formatting).
- `menu.py` is the user-facing interface layer that ties everything together.
- `reports.py`, `budget.py`, `exporter.py`, and `charts.py` each isolate one Standard/Advanced feature, so they can be read independently.

3.3 Module-by-Module Explanation

`expense.py` — The Expense Class

Defines the `Expense` class with four attributes: `amount`, `category`, `date`, and `description`. Includes a `__str__` method for readable printing, and `to_row()` / `from_row()` methods that convert an `Expense` to and from a CSV row — this keeps all the CSV-formatting knowledge in one place rather than scattered across the app.

```
class Expense:
    def __init__(self, amount, category, date, description):
        self.amount = float(amount)
```

```

        self.category = category
        self.date = date
        self.description = description

    def __str__(self):
        return f"{self.date} | {self.category}: ₹{self.amount:.2f} - {self.description}"

```

file_manager.py — Persistence Layer

Uses Python's built-in csv module to read and write data/expenses.csv. `load_expenses()` returns an empty list (instead of crashing) if the file doesn't exist yet — the app's first line of error handling. `save_expenses()` rewrites the whole file after every change. `backup_expenses()` copies the current file into data/backups/ with a timestamped filename.

utils.py — Input Validation

Each `get_valid_*`() function loops, re-prompting the user until they provide valid input: a positive number for amount, one of five fixed options for category, and a correctly formatted YYYY-MM-DD string for date. This is the core of the app's error handling for user input — invalid data can never reach the Expense object.

menu.py — Command-Line Interface

Displays the menu, reads the user's choice, and dispatches to the right function using a dictionary of actions (mapping menu numbers to functions) inside a while True loop. This keeps the dispatch logic short and makes it easy to add a new menu option — just add one line to the actions dictionary.

reports.py — Reporting and Analysis (Standard)

`category_summary()` uses `collections.defaultdict` to group expenses by category, computing the total, average, and count for each. `monthly_report()` filters expenses down to a given year/month using `datetime.strptime()` and reuses `category_summary()` for the per-category breakdown within that month.

budget.py — Budget Planning (Advanced)

Lets the user set a spending limit per category, saved to data/budgets.csv. `check_budget_status()` compares actual spending (via `reports.category_summary()`) against each saved limit. `warn_if_over_budget()` is called immediately after adding a new expense, so the user gets real-time feedback rather than having to check a separate report.

exporter.py — Data Export (Advanced)

`export_to_json()` writes all expenses to a timestamped .json file using Python's built-in json module. `export_to_excel()` writes them to a formatted .xlsx file using the openpyxl package, with a bold header row and sensible column widths.

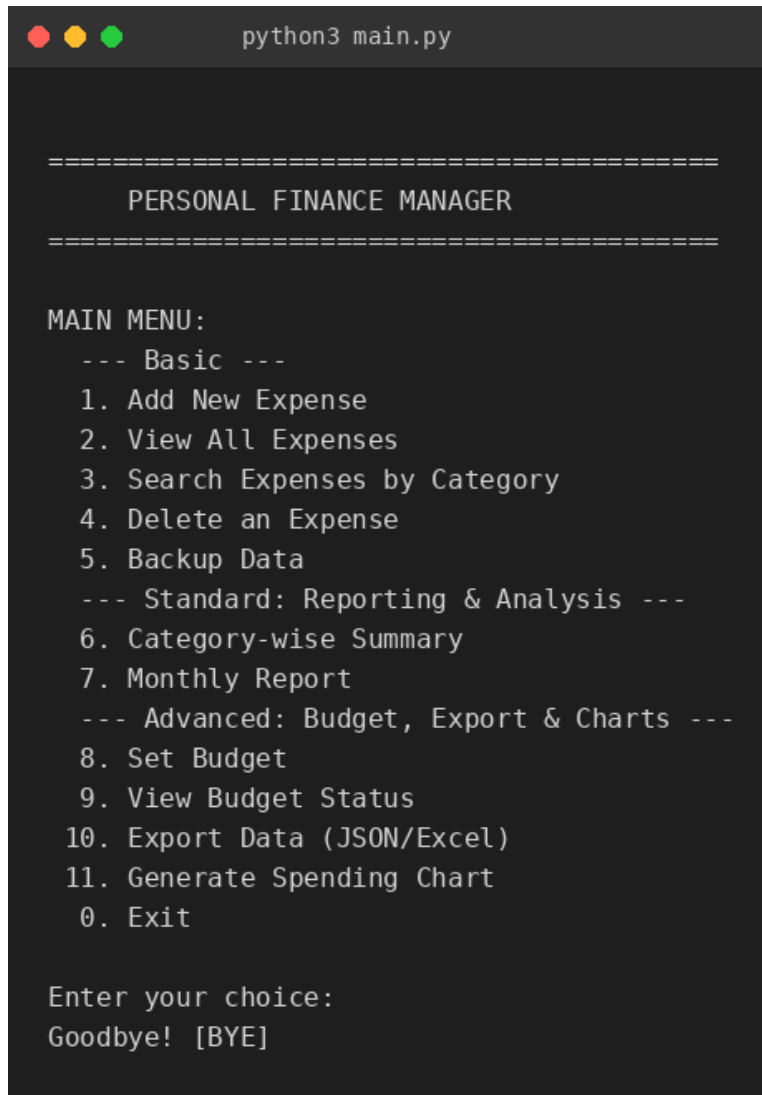
charts.py — Chart Generation (Advanced)

Uses matplotlib in non-interactive ("Agg") mode to draw a bar chart of total spending per category and save it as a PNG file under reports/. A command-line app has no window to display a chart in, so the chart is saved as an image the user opens separately.

4. Screenshots of Working Application

The screenshots below were captured by running the application in a terminal (python3 main.py) and are shown in the order a typical session would follow.

4.1 Main Menu



```
python3 main.py

=====
PERSONAL FINANCE MANAGER
=====

MAIN MENU:
  --- Basic ---
  1. Add New Expense
  2. View All Expenses
  3. Search Expenses by Category
  4. Delete an Expense
  5. Backup Data
  --- Standard: Reporting & Analysis ---
  6. Category-wise Summary
  7. Monthly Report
  --- Advanced: Budget, Export & Charts ---
  8. Set Budget
  9. View Budget Status
 10. Export Data (JSON/Excel)
 11. Generate Spending Chart
  0. Exit

Enter your choice:
Goodbye! [BYE]
```

Figure 1 — The main menu, showing all Basic, Standard, and Advanced options.

4.2 Adding an Expense (Basic)

```
python3 main.py

=====
PERSONAL FINANCE MANAGER
=====

MAIN MENU:
-- Basic --
1. Add New Expense
2. View All Expenses
3. Search Expenses by Category
4. Delete an Expense
5. Backup Data
-- Standard: Reporting & Analysis --
6. Category-wise Summary
7. Monthly Report
-- Advanced: Budget, Export & Charts --
8. Set Budget
9. View Budget Status
10. Export Data (JSON/Excel)
11. Generate Spending Chart
0. Exit

Enter your choice:
ADD NEW EXPENSE:
Enter amount: Enter category (Food/Transport/Entertainment/Shopping/Other): Enter date (YYYY-MM-DD): Enter description:
[OK] Expense added successfully!

Press Enter to continue...
```

Figure 2 — Adding a new expense; the app confirms success and flags it if the category is now over budget.

4.3 Viewing All Expenses (Basic)

```
python3 main.py

=====
PERSONAL FINANCE MANAGER
=====

MAIN MENU:
  --- Basic ---
  1. Add New Expense
  2. View All Expenses
  3. Search Expenses by Category
  4. Delete an Expense
  5. Backup Data
  --- Standard: Reporting & Analysis ---
  6. Category-wise Summary
  7. Monthly Report
  --- Advanced: Budget, Export & Charts ---
  8. Set Budget
  9. View Budget Status
  10. Export Data (JSON/Excel)
  11. Generate Spending Chart
  0. Exit

Enter your choice:
ALL EXPENSES:
1. 2024-01-15 | Food: ₹1500.00 - Grocery shopping
2. 2024-01-16 | Transport: ₹800.00 - Cab to office
3. 2024-01-18 | Entertainment: ₹600.00 - Movie night
4. 2024-01-20 | Shopping: ₹2200.00 - New shoes
5. 2024-01-22 | Food: ₹450.00 - Lunch with friends
6. 2024-03-10 | Food: ₹1200.00 - Weekly groceries

Total spent: ₹6750.00 | Number of expenses: 6

Press Enter to continue...
```

Figure 3 — Viewing all recorded expenses with a running total.

4.4 Category-Wise Summary (Standard)

```
python3 main.py

=====
PERSONAL FINANCE MANAGER
=====

MAIN MENU:
  --- Basic ---
  1. Add New Expense
  2. View All Expenses
  3. Search Expenses by Category
  4. Delete an Expense
  5. Backup Data
  --- Standard: Reporting & Analysis ---
  6. Category-wise Summary
  7. Monthly Report
  --- Advanced: Budget, Export & Charts ---
  8. Set Budget
  9. View Budget Status
  10. Export Data (JSON/Excel)
  11. Generate Spending Chart
  0. Exit

Enter your choice:
CATEGORY-WISE SUMMARY:
Category          Total (₹)    Average (₹)    Count
-----
Food              3150.00      1050.00         3
Shopping          2200.00      2200.00         1
Transport          800.00       800.00          1
Entertainment      600.00       600.00          1
-----
GRAND TOTAL       6750.00

Press Enter to continue...
```

Figure 4 — Category-wise summary showing totals, averages, and counts.

4.5 Monthly Report (Standard)

```
python3 main.py

=====
PERSONAL FINANCE MANAGER
=====

MAIN MENU:
  --- Basic ---
  1. Add New Expense
  2. View All Expenses
  3. Search Expenses by Category
  4. Delete an Expense
  5. Backup Data
  --- Standard: Reporting & Analysis ---
  6. Category-wise Summary
  7. Monthly Report
  --- Advanced: Budget, Export & Charts ---
  8. Set Budget
  9. View Budget Status
 10. Export Data (JSON/Excel)
 11. Generate Spending Chart
  0. Exit

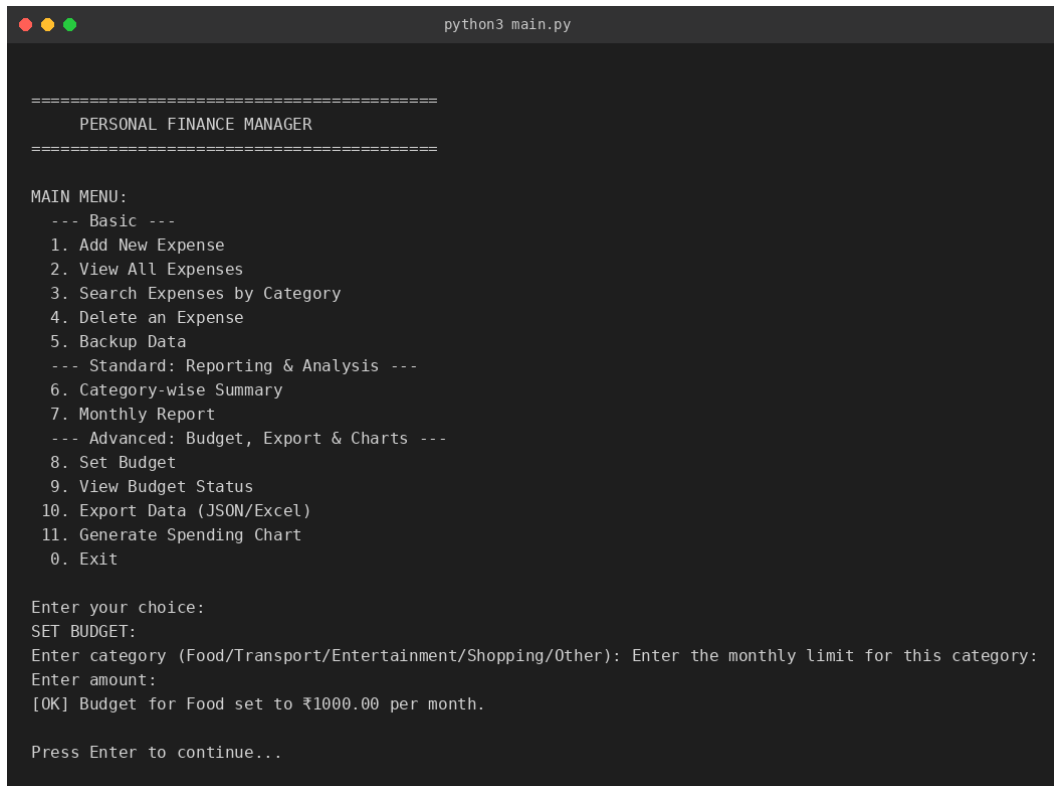
Enter your choice:
Enter year (e.g. 2024): Enter month (1-12):
MONTHLY REPORT – January 2024
-----
2024-01-15 | Food: ₹1500.00 - Grocery shopping
2024-01-16 | Transport: ₹800.00 - Cab to office
2024-01-18 | Entertainment: ₹600.00 - Movie night
2024-01-20 | Shopping: ₹2200.00 - New shoes
2024-01-22 | Food: ₹450.00 - Lunch with friends
-----
Total spent in January 2024: ₹5550.00
Number of transactions: 5

By category:
Shopping: ₹2200.00 (1 transactions)
Food: ₹1950.00 (2 transactions)
Transport: ₹800.00 (1 transactions)
Entertainment: ₹600.00 (1 transactions)

Press Enter to continue...
```

Figure 5 — Monthly report for January 2024, with a per-category breakdown.

4.6 Setting a Budget (Advanced)



```
python3 main.py

=====
PERSONAL FINANCE MANAGER
=====

MAIN MENU:
--- Basic ---
1. Add New Expense
2. View All Expenses
3. Search Expenses by Category
4. Delete an Expense
5. Backup Data
--- Standard: Reporting & Analysis ---
6. Category-wise Summary
7. Monthly Report
--- Advanced: Budget, Export & Charts ---
8. Set Budget
9. View Budget Status
10. Export Data (JSON/Excel)
11. Generate Spending Chart
0. Exit

Enter your choice:
SET BUDGET:
Enter category (Food/Transport/Entertainment/Shopping/Other): Enter the monthly limit for this category:
Enter amount:
[OK] Budget for Food set to ₹1000.00 per month.

Press Enter to continue...
```

Figure 6 — Setting a monthly budget limit for the Food category.

4.7 Budget Status (Advanced)

```
python3 main.py

=====
PERSONAL FINANCE MANAGER
=====

MAIN MENU:
  --- Basic ---
  1. Add New Expense
  2. View All Expenses
  3. Search Expenses by Category
  4. Delete an Expense
  5. Backup Data
  --- Standard: Reporting & Analysis ---
  6. Category-wise Summary
  7. Monthly Report
  --- Advanced: Budget, Export & Charts ---
  8. Set Budget
  9. View Budget Status
  10. Export Data (JSON/Excel)
  11. Generate Spending Chart
  0. Exit

Enter your choice:
BUDGET STATUS:
Category          Spent (₹)   Budget (₹)   Status
-----
Food              5150.00    1000.00     [!] OVER

Press Enter to continue...
```

Figure 7 — Budget status view, flagging categories that are over their limit.

4.8 Exporting Data (Advanced)

```
python3 main.py

=====
PERSONAL FINANCE MANAGER
=====

MAIN MENU:
--- Basic ---
1. Add New Expense
2. View All Expenses
3. Search Expenses by Category
4. Delete an Expense
5. Backup Data
--- Standard: Reporting & Analysis ---
6. Category-wise Summary
7. Monthly Report
--- Advanced: Budget, Export & Charts ---
8. Set Budget
9. View Budget Status
10. Export Data (JSON/Excel)
11. Generate Spending Chart
0. Exit

Enter your choice:
EXPORT DATA:
1. Export as JSON
2. Export as Excel (.xlsx)
Choose an option (1-2):
[OK] Exported to: ./data/exports/expenses_20260923_035722.json

Press Enter to continue...
```

Figure 8 — Exporting all expense data to a JSON file.

4.9 Generated Spending Chart (Advanced)

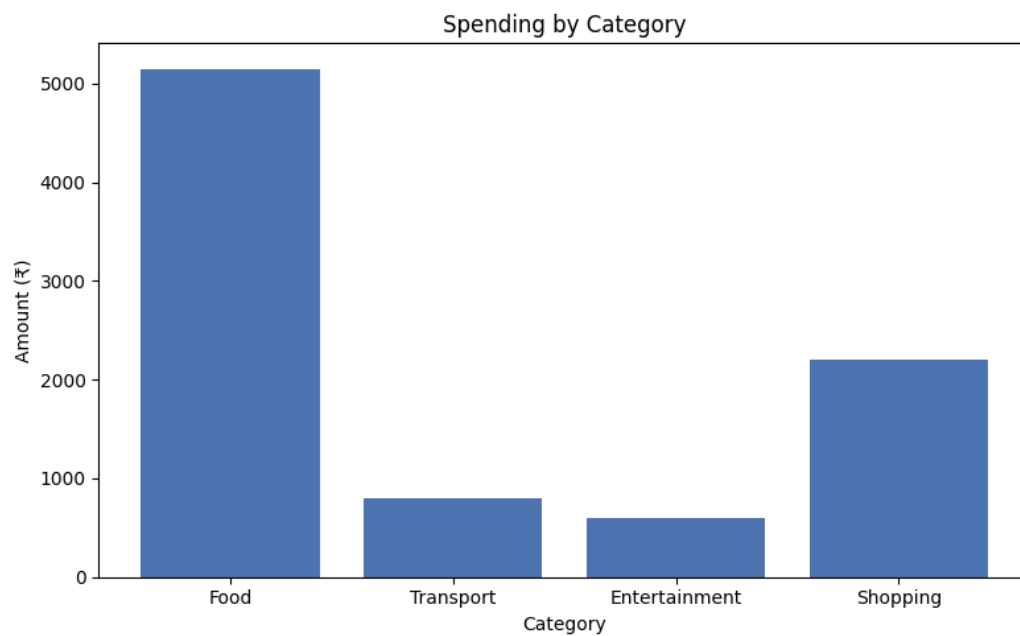


Figure 9 — Auto-generated bar chart of spending by category, saved as a PNG file.

5. How Technical Requirements Were Met

The table below maps each technical requirement from the project brief to the specific implementation that satisfies it.

Technical Requirement	How It Was Met	File(s)
Create Expense class with attributes: amount, category, date, description	Implemented exactly as specified in the Expense class, with a constructor storing all four attributes and amount cast to float for safe arithmetic.	src/expense.py
Implement file operations using the CSV module for data persistence	load_expenses() and save_expenses() use Python's built-in csv.reader / csv.writer to read and write data/expenses.csv on every change.	src/file_manager.py
Add comprehensive error handling for all user inputs	Dedicated validators re-prompt on invalid amounts, categories, and dates; file operations are wrapped in try/except so a missing or unreadable file never crashes the app.	src/utils.py, src/file_manager.py
Create interactive command-line interface with menu system	A numbered menu loop reads the user's choice and dispatches it via an actions dictionary, looping until the user chooses Exit.	src/menu.py
Implement data validation and formatting functions	get_valid_amount(), get_valid_category(), and get_valid_date() validate input; Expense.__str__() formats output consistently.	src/utils.py, src/expense.py
Organize code into modular structure (separate modules for classes, file handling, UI)	Code is split across 8 focused modules (expense, file_manager, utils, menu, reports, budget, exporter, charts) instead of one script.	src/*.py
Include report generation with calculations (total, average, category-wise expenses)	category_summary() computes totals, averages, and counts per category; monthly_report() adds date-based filtering.	src/reports.py
Add data backup and restore functionality	backup_expenses() copies the CSV file to data/backups/ with a timestamped filename on demand.	src/file_manager.py
Advanced: budget planning	Category budgets are set and saved to data/budgets.csv; the app warns immediately if a new expense pushes a category over budget, and a dedicated status view shows all categories at a glance.	src/budget.py
Advanced: data export	Expenses can be exported to timestamped .json or formatted .xlsx files on demand.	src/exporter.py
Advanced: charts	A bar chart of category-wise spending is generated with matplotlib and saved as a PNG image.	src/charts.py

6. User Manual

6.1 Starting the App

From the project's root folder, run:

```
python3 main.py
```

6.2 Menu Options

Option	What It Does
1. Add New Expense	Enter amount, category, date (YYYY-MM-DD), and description. The app re-prompts on invalid input, and warns immediately if you've now gone over a set budget for that category.
2. View All Expenses	Lists every expense with a running total.
3. Search Expenses by Category	Filters to one category, with a subtotal.
4. Delete an Expense	Pick a numbered expense to remove, or 0 to cancel.
5. Backup Data	Saves a timestamped copy of expenses.csv to data/backups/.
6. Category-wise Summary	Shows total, average, and count of expenses per category, sorted by highest spending.
7. Monthly Report	Enter a year and month; view every transaction that month plus a per-category breakdown.
8. Set Budget	Choose a category and set a monthly spending limit for it. Limits persist between runs.
9. View Budget Status	Shows spending vs. budget for every category with a limit, marked OK or OVER.
10. Export Data (JSON/Excel)	Export all expenses to a .json or .xlsx file, saved to data/exports/.
11. Generate Spending Chart	Creates a bar chart (PNG) of spending per category, saved to reports/.
0. Exit	Closes the program. Data is saved after every change, so nothing extra is needed before exiting.

6.3 Tips

- Dates must be entered as YYYY-MM-DD (e.g. 2024-03-05).
- Categories are not case-sensitive — 'food' and 'FOOD' both work.
- Budgets are per category, applied against the all-time total in that category.
- The raw data file (data/expenses.csv) can be opened directly in Excel, but should only be edited through the app to avoid formatting mistakes.

7. Troubleshooting Guide

Problem	Solution
ModuleNotFoundError: No module named 'menu'	Run main.py from the project's root folder, not from inside src/.
"Charts need the 'matplotlib' package"	Run: pip install -r requirements.txt
"Excel export needs the 'openpyxl' package"	Run: pip install -r requirements.txt
Expenses don't appear after restarting the app	Check that data/expenses.csv exists and wasn't deleted, renamed, or moved.
"Invalid date format" keeps appearing	Use exactly YYYY-MM-DD, e.g. 2024-01-15 (not DD-MM-YYYY or MM/DD/YYYY).
"Invalid choice" on a category prompt	Category must be one of: Food, Transport, Entertainment, Shopping, Other (not case-sensitive).
Budget warning never appears	Set a budget for that category first via option 8 — warnings only apply to categories with a saved limit.
Chart or export file doesn't seem to save	Confirm the app has permission to create folders (reports/, data/exports/) in the project directory.

7.1 Testing Evidence

Every menu option was manually tested end-to-end before this documentation was written:

- Add/view/search/delete tested with both valid and invalid input (bad numbers, bad dates, empty descriptions) — re-prompting confirmed working.
- Category summary and monthly report totals verified against manual calculation.
- A budget was set, then an expense was added that exceeded it — the over-budget warning appeared immediately, and Budget Status reflected it correctly.
- Data was exported to both JSON and Excel and confirmed to open correctly with all four fields intact.
- A chart was generated and confirmed to show correct per-category bars.
- Backup was run and confirmed to produce a timestamped CSV copy in data/backups/.